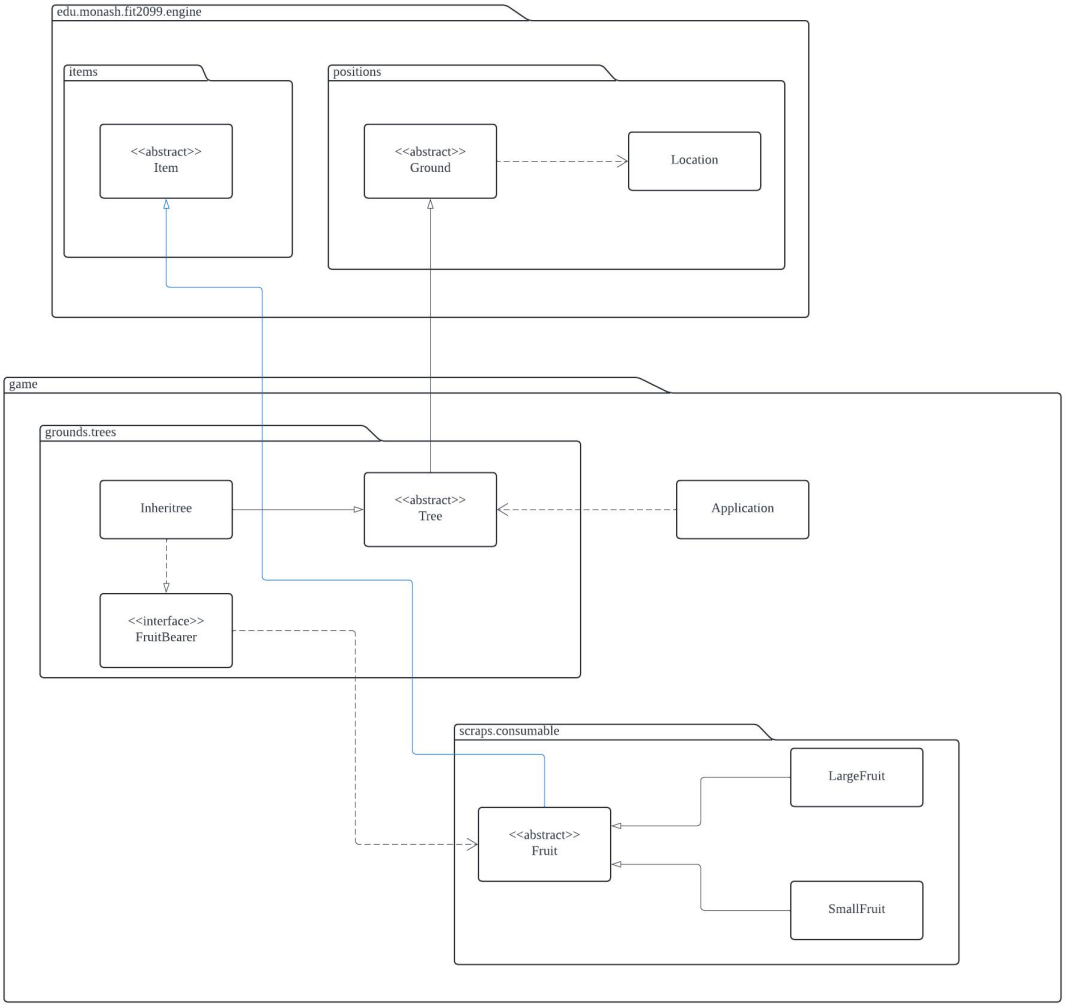


This diagram represents an object-oriented design of Assignment 1 Requirement 2 which aims to introduce a type of plant name “Inheritree” which can drop different size of fruit based on its maturity. It consists of 2 abstract classes (Tree, Fruit) in the game package which inherits the abstract classes (Item, Ground) from the game engine provided. A concrete class (Inheritree) which implements an interface (FruitBearer) and inherits one of the abstract class (Tree). Two concrete classes (LargeFruit, SmallFruit) inherits the other abstract class (Fruit).

The object-oriented design principles applied in this design is:

- Single Responsibility Principle
- Open-Closed Principle
- Liskov Substitution Principle
- Dependency Inversion Principle

How are the principles being applied?

- Each classes have their own specific responsibility. For example, Tree class represents tree objects in the game and Fruit class represents the fruit items in the game. Inheritree class represents a specific type of tree called Inheritree

with the capability of bearing fruit. LargeFruit and SmallFruit classes are responsible for representing different sizes of fruit produced by Inheritree object. This promotes code maintainability as modifying one aspect of the system are less likely to impact the other unrelated parts. (Single Responsibility Principle)

- The design is open for extension without requiring the modification of existing code. For example, a new type of tree such as “PalmTree” can be added just by creating the class and extends it using Tree abstract class. The usage of abstraction allows new implementation without altering the existing Tree abstract class. This promotes extensibility. (Open-Closed Principle)

- The current design allows objects of subtypes such as LargeFruit and Inheritree to be able to be substituted by their base types such as Tree and Fruit without affecting the correctness of the code which allows polymorphic behaviour, meaning LargeFruit object can be treated as Fruit object without affecting the behaviour of operations that depends on their type. (Liskov Substitution Principle)

- The Inheritree produces new Fruit objects using the interface implemented which reduces the direct dependencies of Inheritree with these fruit classes. It ensures that no concrete classes depending on each other and should only depend on abstractions. This approach promotes flexibility as there are no tight coupling between the classes. (Dependency Inversion Principle)

The pros and cons in the system:

Pros:

- Adhere SOLID principles which promotes code readability, flexibility and maintainability.
- Reduced dependencies and coupling between the classes means reduce the risk of bugs occurring when the code is being modified.
- Polymorphism is allowed and able to treat the objects uniformly, enabling interchangeable use of object type.
- Introduction of interface reduces unnecessary dependencies and allow multiple inheritance.

Cons:

- As the system scales bigger, the design can be over complex.
- It is difficult to maintain large number of classes over time as the design extended.
- Might have potential of tight coupling if concrete classes rely heavily on the implementation details of other classes.

Let’s extend the design by introducing two new tree classes such as “Birch Tree” and “Banana Tree”, each representing a tree that does not bear a fruit and a tree that bears a fruit object (Banana) in the game. Similar to Inheritree, These two concrete classes inherit from the Tree abstract class. Since BananaTree class is the only tree that capable of bearing fruit among it, it has to implement the FruitBearer interface while BirchTree doesn’t. Banana class is introduced as a type of fruit so it must inherits Fruit abstract class just like SmallFruit and LargeFruit class.

When the Application runs, as the game ticks, all the tree classes will grow into their mature form when the certain amount of ticks are passed due to being inherited from the same Tree abstract class. BirchTree does not possess the ability of producing fruit due to not implementing FruitBearer interface, so it will remain there forever until the game ends. While the other BananaTree will drop Banana object around the tree using produceFruit() from the FruitBearer interface with a certain spawn rate just like Inheritree with its SmallFruit and LargeFruit.

Based on the description of the potential extension above, this OOP design is shown to seamlessly integrate the new features by leveraging the inheritance and inteface implementation. It also provides a solid foundation as the game continues to develop and be enhanced.